📘 ASSIGNMENT DOCUMENT (FINAL STUDENT VERSION)

Building an Improved Agentic Data Pipeline in n8n

(Complete Implementation – With All Code Included)

1️⃣ Assignment Objective

You will build a complete production-style workflow that:

1. Reads CSV data
2. Profiles the dataset
3. Uses AI for data quality analysis
4. Applies decision logic
5. Cleans data with JavaScript
6. Upserts into MySQL
7. Runs SQL analytics
8. Restructures output
9. Generates an executive AI report
10. Saves the report as a text file

This is a real-world hybrid pipeline: AI + Rules + Database + Automation.

2️⃣ Dataset Requirement

Create:

Employee.csv

Columns:

employee_id,name,department,age,salary,joining_year

Include:

- Minimum 20 rows
- 1 missing department
- 1 missing age
- 1 invalid salary (example: not_available)
- Optional: one negative value

3️⃣ Step-by-Step Implementation

🔹 STEP 1 — Manual Trigger

Add:

Manual Trigger

🔹 STEP 2 — Read File From Disk

Node:

Read File From Disk

Select your CSV file.

🔹 STEP 3 — Extract From CSV

Node:

Extract From CSV

This converts rows into JSON.

🔹 STEP 4 — Prepare Data for Analysis

Add Code Node Name:

Prepare Data for Analysis

Paste this EXACT code:

```
const allRows = items.map(item => item.json);

const rowCount = allRows.length;

const columns = Object.keys(allRows[0] || {});

let missingStats = {};

let typeStats = {};

let basicStats = {};

columns.forEach(col => {

 let values = allRows.map(r => r[col]);

 let missing = values.filter(v => v === null || v === "" || v === undefined).length;

 missingStats[col] = ((missing / rowCount) * 100).toFixed(2);

 let numericCount = values.filter(v => !isNaN(parseFloat(v))).length;

 typeStats[col] = {

  numeric_percentage: ((numericCount / rowCount) * 100).toFixed(2)

 };

 let negatives = values.filter(v => !isNaN(v) && Number(v) < 0).length;

 basicStats[col] = {

  negative_values: negatives
```

```
  };
});
return [
  {
    json: {
      row_count: rowCount,
      columns: columns,
      missing_percentage: missingStats,
      type_analysis: typeStats,
      anomaly_signals: basicStats,
      sample_rows: allRows.slice(0, 5)
    }
  }
];
```

This prepares a summary for the AI.

🔹 STEP 5 — Data Quality Agent

Add OpenAI Node Name:

Data Quality Agent

Model:

gpt-4.1-mini

System Prompt:

You are a Data Quality Analysis Agent.

IMPORTANT RULES:

1. Only refer to provided column names.

2. Do not invent new columns.

3. If insufficient evidence exists, state: "Not detectable from provided data".

4. Return structured JSON only.

Dataset summary:

{{ JSON.stringify($json) }}

Return strictly in this format:

```
{
  "missing_values_analysis": {},
  "data_type_analysis": {},
  "anomaly_detection": {}
}
```

🔹 STEP 6 — Analysis Summary

Add Code Node Name:

Analysis Summary

Paste this EXACT code:

```
const rawText = $json.output?.[0]?.content?.[0]?.text;
```

```
if (!rawText) {
  return [{ json: { error: "OpenAI text not found", received: $json } }];
}
let aiOutput;
try {
  aiOutput = JSON.parse(rawText);
} catch (err) {
  return [{ json: { error: "JSON parse failed", rawText } }];
}
if (!aiOutput.missing_values_analysis) {
  return [{ json: { error: "missing_values_analysis not found", aiOutput } }];
}
const missingValues = aiOutput.missing_values_analysis;
let maxMissing = 0;
for (const col in missingValues) {
  const percent = parseFloat(String(missingValues[col]).replace("%", ""));
  if (!isNaN(percent) && percent > maxMissing) {
    maxMissing = percent;
  }
}
return [
```

```
 {
   json: {
     max_missing_percentage: maxMissing,
     missing_values: missingValues
   }
 }
];
```

🔹 STEP 7 — Decision Layer

Add IF Node Name:

Decide Next Step

Condition:

{{$json.max_missing_percentage}} > 20

TRUE → Stop FALSE → Continue

🔹 STEP 8 — Stop Node

Add Set Node Name:

Stop Here

JSON Output:

```
{
  "workflow_status": "Stopped",
```

```
  "reason": "Missing values exceed acceptable threshold"
}
```

🔹 STEP 9 — Cleaning Layer

Add Code Node Name:

Code in JavaScript

Paste this EXACT code:

```
const originalItems = $items("Extract From CSV");
let cleaned = [];
for (const item of originalItems) {
  const row = item.json;
  if (row.age === null || row.age === undefined || String(row.age).trim() === "") {
    continue;
  }
  if (isNaN(parseFloat(row.salary))) {
    continue;
  }
  row.employee_id = parseInt(row.employee_id);
  row.age = parseInt(row.age);
  row.salary = parseFloat(row.salary);
  row.joining_year = parseInt(row.joining_year);
```

```
  cleaned.push({ json: row });
}

return cleaned;
```

This ensures:

- No missing age
- No invalid salary
- Proper numeric types

🔹 STEP 10 — MySQL Table

Execute:

```
CREATE TABLE employees (
  employee_id INT PRIMARY KEY,
  name VARCHAR(100),
  department VARCHAR(50),
  age INT,
  salary FLOAT,
  joining_year INT
);
```

🔹 STEP 11 — Upsert Node

Operation:

Upsert

Match Column:

employee_id

🔹 STEP 12 — SQL Queries

Query 1

SELECT

COUNT(*) AS total_employees,

ROUND(AVG(salary),2) AS average_salary,

MAX(salary) AS highest_salary,

MIN(salary) AS lowest_salary,

ROUND(AVG(age),1) AS average_age

FROM employees;

Query 2

SELECT department, COUNT(*) AS employee_count

FROM employees

GROUP BY department;

🔹 STEP 13 — Flatten the Data

Add Code Node Name:

Flatten the data

Paste this EXACT code:

```
const data = $input.first().json;

const overview = data.overview;

const deptArray = data.department_distribution;

let dept = {};

deptArray.forEach(item => {

  const key = Object.keys(item)[0];

  dept[key] = item[key];

});

return [{

  json: {

    overview,

    department_distribution: dept

  }

}];
```

🔹 STEP 14 — Reporting Agent

Model:

gpt-5

Prompt:

You are a senior data analyst.

Generate a professional plain-text executive report.

Structure:

WORKFORCE OVERVIEW

SALARY INSIGHTS

DEPARTMENT DISTRIBUTION

KEY OBSERVATIONS

RECOMMENDATIONS

Do not use markdown.

Do not return JSON.

Overview:

{{JSON.stringify($json.overview)}}

Department Distribution:

{{JSON.stringify($json.department_distribution)}}

🔹 STEP 15 — Convert to Text File

Add Code Node Name:

Convert to Text File

Paste this EXACT code:

```
let report =
  $json.report ||
  $json.content ||
  $json.output?.[0]?.content?.[0]?.text;
```

```
if (!report) {
  return [{
    json: {
      error: "Report text not found",
      available_keys: Object.keys($json),
      full_data: $json
    }
  }];
}
return [{
  binary: {
    data: {
      data: Buffer.from(report, 'utf8').toString('base64'),
      mimeType: 'text/plain',
      fileName: `Employee_Report_${new Date().toISOString().slice(0,10)}.txt`
    }
  }
}];
```

---

4 Submission Requirements

Students must submit:

1. Full workflow screenshot
2. MySQL table screenshot
3. Generated text report file
4. One-page explanation covering:
    - Why AI is used for interpretation
    - Why rule-based cleaning is required
    - Why decision logic protects workflow
    - Why upsert is better than insert

This version now gives students:

- Zero ambiguity
- All JavaScript included
- Complete SQL included
- Exact OpenAI prompts
- Production-style architecture
